Merge Sort

Merge sort is a divide-and-conquer algorithm that :

- Divides the input array into two halves.
- Recursively sort each half.
- And the merges the sorted halves into a single sorted array.

→ // C program

```
#include<stdio.h>
// merge two sorted halves into one sorted array
void merge (int arr[], int left, int mid, int
            right)
{
    int i = left;      // starting index of left subarray
    int j = mid+1;     // starting index of right subarray
    int k = 0;         // index of temporary array
// temporary array to hold merged result
    int temp[right - left +1];

// Compare elements from both halves and store the smaller
one

while (i<=mid && j<=right)
    {
```

Akash

```
if(arr[i] <arr[j] )
  {
      temp[k++]= arr [i++];
  }
else
  {
      temp[k++]= arr [j++];
  }
}

// copy remaining element from left half
while (i<=mid)
  {
    temp[k++]= arr [i++];
  }

// copy remaining element from right half
while( j<=right)
  {
    temp[k++]= arr[j++];
  }

// copy merged element back into original array
for (i= left, k=0; i<=right ; i++, k++)
  {
```

Akash

```
        arr[i] = temp[k];
    }

// Recursive function to divide array and call merge
void mergesort(int arr[], int left, int right)
    {
        if(left < right)
        {
            int mid = (left + right) / 2;
        mergesort(arr, left, mid); // sort left half
        mergesort(arr, mid+1, right); // sort right half
        merge(arr, left, mid, right); //merge both halves
        }
    }

int main()
    {
    int arr[20], n, i;
    printf("enter size of an array");
    scanf("%d", &n);
    for(i=0; i<n; i++)
        {
```

```
    printf("enter data\n");
    scanf("%d", &arr[i]);
}

// perform mergesort

mergesort(arr, 0, n-1);

printf("Sorted array:\n");
for(i=0;i<n;i++)
  {
      printf("%d  ", arr[i]);
  }
return 0;
}
```

Akash

→ Algorithm:-

Step 1:- Start

Step 2:- If the array has more than one element

2.1.) Find the middle index of the array.

2.2.) Divide the array into two halves: left and right

2.3.) Recursively apply merge sort to both halves

2.4.) Merge the two sorted halves into a single sorted array

Step 3.) Stop

Akash

Eg;

				mid ↑					
10	15	50	17	20	25	30	16	70	6

left = 0

right = n-1

= 9

mid = 0+9/2 = 4

left sub array → [10 | 15 | 50 | 17 | 20] (left, mid, right)

Right sub array → [25 | 30 | 16 | 70 | 6] (left, mid, right)

left = 0

right = mid = 4

mid = 0+4/2 = 2

left = mid+1

= 5

right = 9

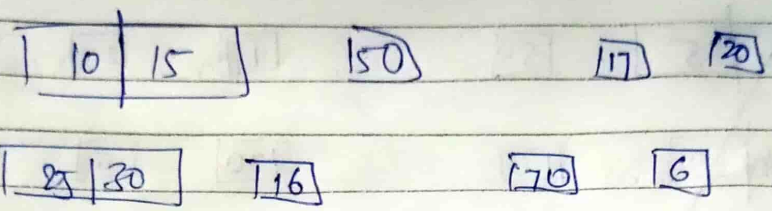
mid = (5+9)/2 = 7

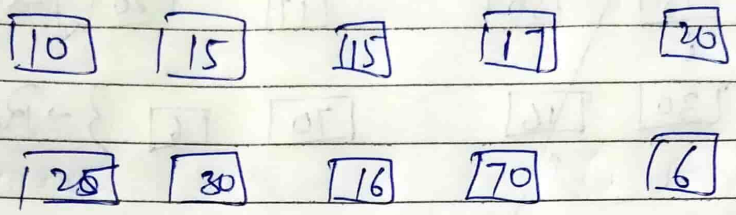
Similary

[10 | 15 | 50] [17 | 20]

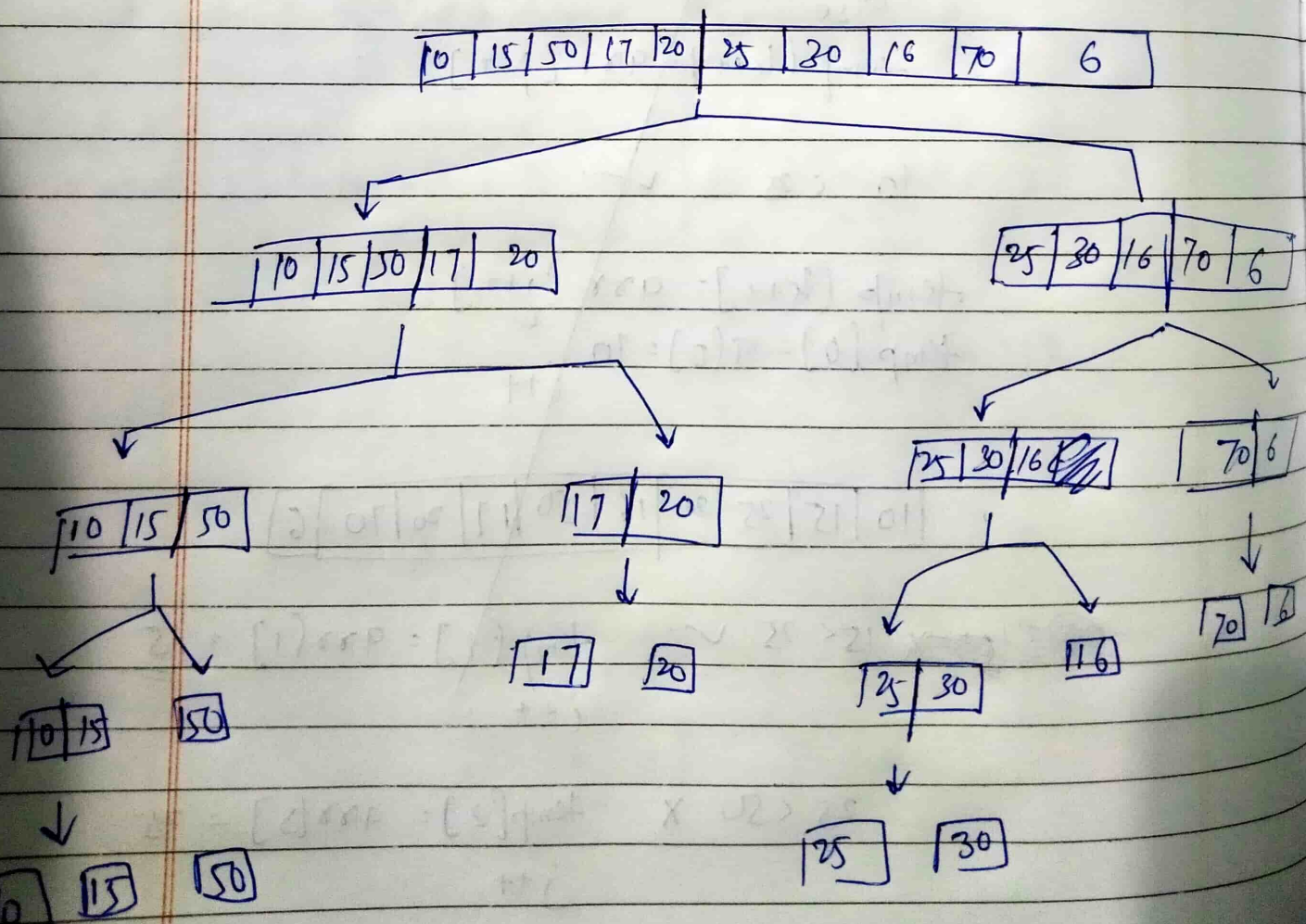
[25 | 30 | 16] [70 | 6]

Akash





Another way



Akash

10 15 50 17 20 25 30

6 70 6

10 15 50 17 20

25 30 16 6 70

10 15 50 17 20

25 30 16 6 70

10 15 17 20 50

6 16 25 30 70

6 10 15 16 17 20 25 30 50 70

Akash

Time Complexity:-

Case	Time Complexity
Best	$O(n \log n)$
Average	$O(n \log n)$
Worst	$O(n \log n)$

Akash

Akash